

COMMUNITY OVER CODE · GLASGOW 2026

# WireGuard for Apache NuttX

A WireGuard VPN implemented as a `wg0` network device — on real hardware, and inside a real NuttX kernel build.

**ESP32-S3 & SPRESENSE**

TUNNEL UP OVER REAL WI-FI

**Real WireGuard peers**

LINUX KERNEL, WINDOWS CLIENT

**rv-virt knetnsh64**

REAL KERNEL BUILD (BUILD\_KERNEL)

**NuttX 13.0.1 / master / 12.7.0**

APPS V0.1.1 · ONE SOURCE

Satoru Akita · Sony Semiconductor Solutions



# Agenda

Seven chapters, about thirty minutes.

---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# Agenda

Seven chapters, about thirty minutes.

---

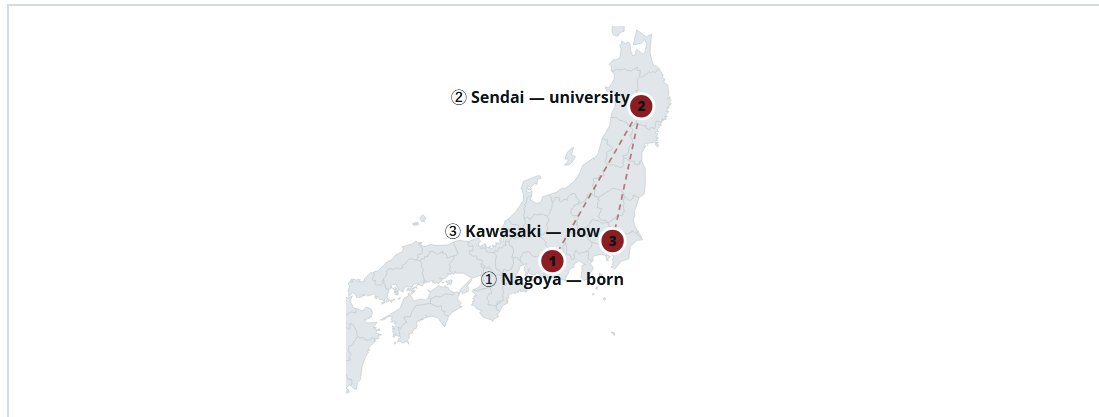
|   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# About Me

From Nagoya, by way of space, robotics, and a lot of conferences.



Satoru Akita — edge AI engineer at Sony Semiconductor Solutions, working with Apache NuttX from the application side.



## UNIVERSITY

Space

Robotics

MEMS

Semiconductors



Hackathon regular



SW conference organizer

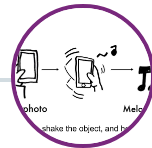


Published an OSS book



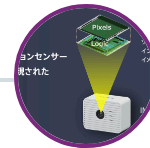
## Robotics

Molecular robotics, BIOMOD world champion (2015)



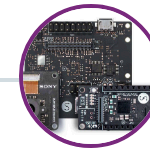
## Computer Vision

AI / image-processing systems engineering



## Edge AI

AITRIOS edge AI cameras



## Embedded Systems

SPRESENSE, embedded app contest — world 3rd

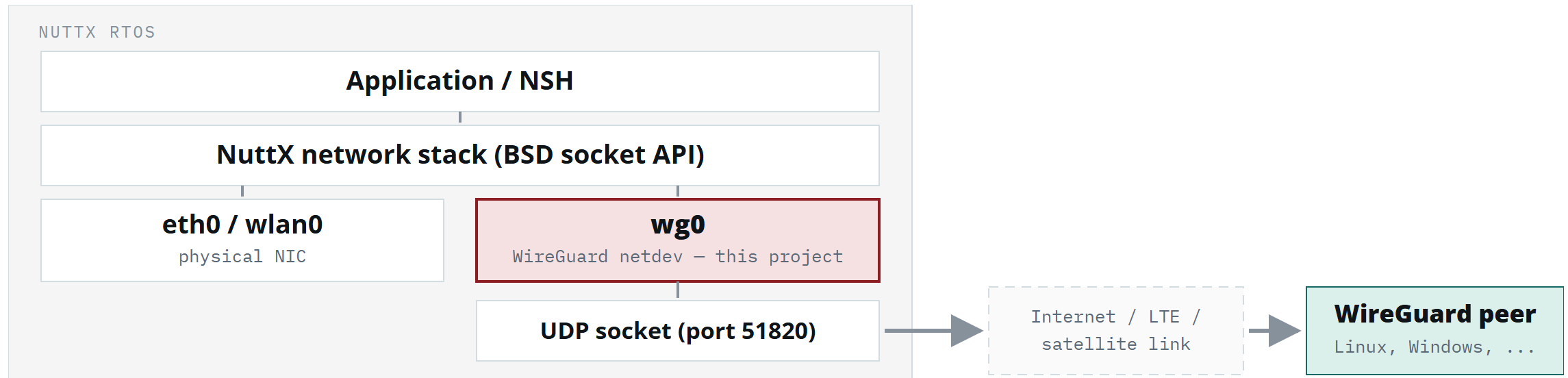


## NuttX / WireGuard

This talk

# Project Overview

This project implements WireGuard as a NuttX network device.

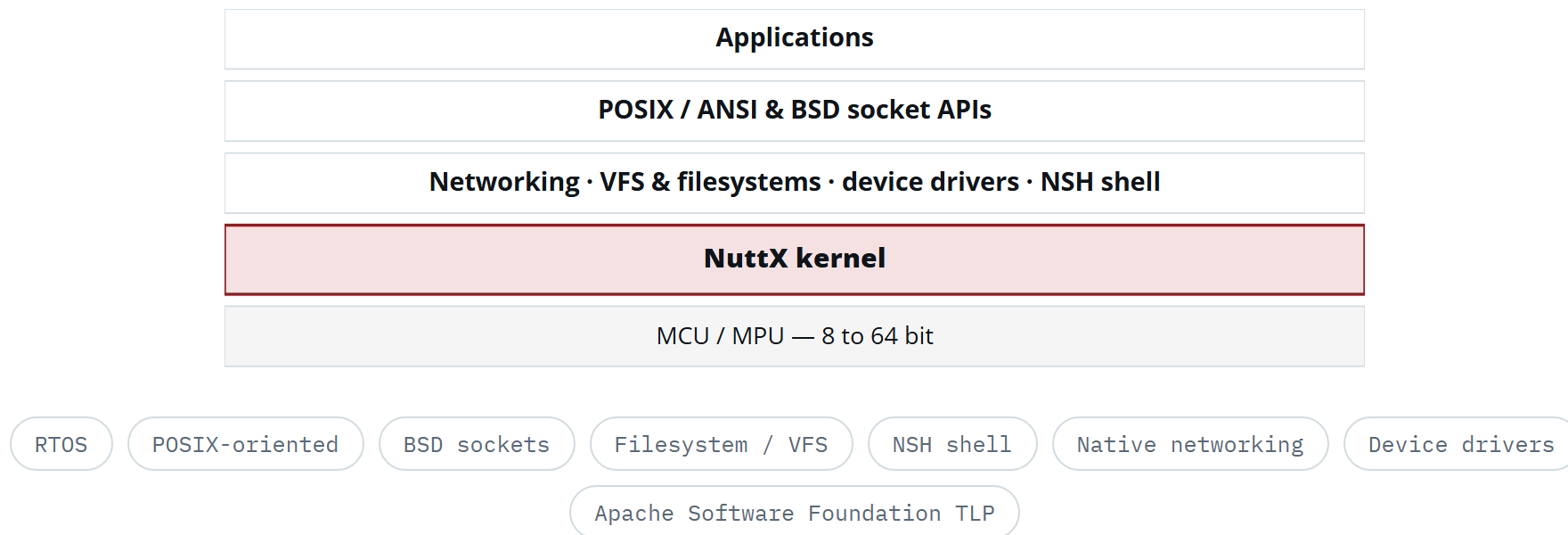


Packets routed to `wg0` are encrypted and sent as UDP; UDP arriving on the port is decrypted and delivered as if it had arrived on `wg0`. The peer can be any existing WireGuard endpoint.

**TAKEAWAY** `wg0` looks like an ordinary interface to applications; only the "wire" underneath is a UDP socket instead of a physical link.

# Background

## What is Apache NuttX?



**TAKEAWAY** NuttX brings a Unix-like programming model — POSIX APIs, BSD sockets, filesystems, and devices — to constrained embedded systems.

# Positioning

Where NuttX fits — and where it is used.

| RTOS            | DESIGN CENTER                        | TYPICAL USE                                     | ARCHITECTURES                  | BOARDS                                                                                          |
|-----------------|--------------------------------------|-------------------------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------|
| <b>FreeRTOS</b> | Lightweight RT kernel                | MCU firmware, control, IoT                      | 15+ CPU architectures          | 300+ supported boards                                                                           |
| <b>Zephyr</b>   | Integrated connected-device platform | Wearables, industrial & connected IoT           |                                |                                                                                                 |
| <b>NuttX</b>    | <b>Unix-like embedded OS</b>         | <b>Networking, filesystems, drones, sensing</b> | 1,500+ configuration templates | IN PRODUCTION<br>Sony · PX4 drones · Xiaomi OpenVela (1,000+ SKUs) · Japan's 2024 lunar mission |

FreeRTOS/FreeRTOS 7.8k★ · zephyrproject-rtos/zephyr 16.5k★ · apache/nuttX 4.1k★

Stars measure developer visibility, not deployment volume.

**TAKEAWAY** NuttX is niche next to FreeRTOS and Zephyr by star count, but it already ships in production hardware, from drones to space missions.

Sources: [nuttx.apache.org](https://nuttx.apache.org) · [nuttx.apache.org/market-share](https://nuttx.apache.org/market-share) · [docs.px4.io](https://docs.px4.io) · GitHub star counts, October 2026

# History

## NuttX at Sony — from audio to edge sensing.

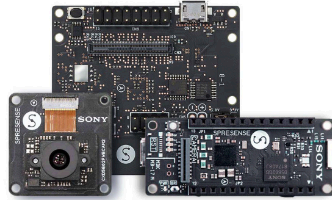
---

- 2015**
    - **Sony audio products**  
NuttX shipping in production audio devices.
  - 2018–19**
    - **SPRESENSE / CXD56xx**  
Audio, GNSS, camera and multicore support land in NuttX; presented at Open Source Summit + ELC Europe 2019.
  - 2020**
    - **Upstream collaboration**  
Sony engineers become major NuttX upstream contributors — “A Journey from Fork to Mainline,” NuttX Online Workshop 2020.
  - 2020s**
    - **Sensing & edge devices**  
Camera, GNSS and sensing use cases broaden across deployed products.
- 
- TAKEAWAY** Sony's use of NuttX predates edge AI; years of shipped audio and SPRESENSE experience paved the way into sensing and camera systems.

Sources: M. Ishikawa (Sony), “NuttX for Embedded Linux Developers,” OSS+ELC Europe 2019 · A. Jerpelea (Sony), “A Journey from Fork to Mainline,” NuttX Online Workshop 2020 · [nuttx.apache.org/docs](https://nuttx.apache.org/docs) (CXD56xx)

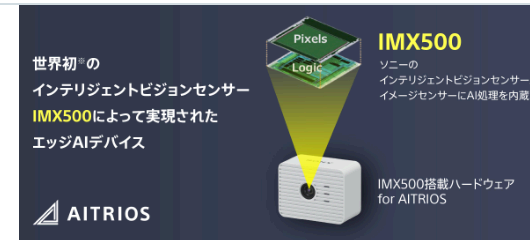
# Experience

## NuttX in SPRESENSE and AITRIOS — my side of it.



### SPRESENSE

NuttX SDK · sensing & camera · GNSS / LTE · prototyping and embedded development.



### AITRIOS / edge AI camera

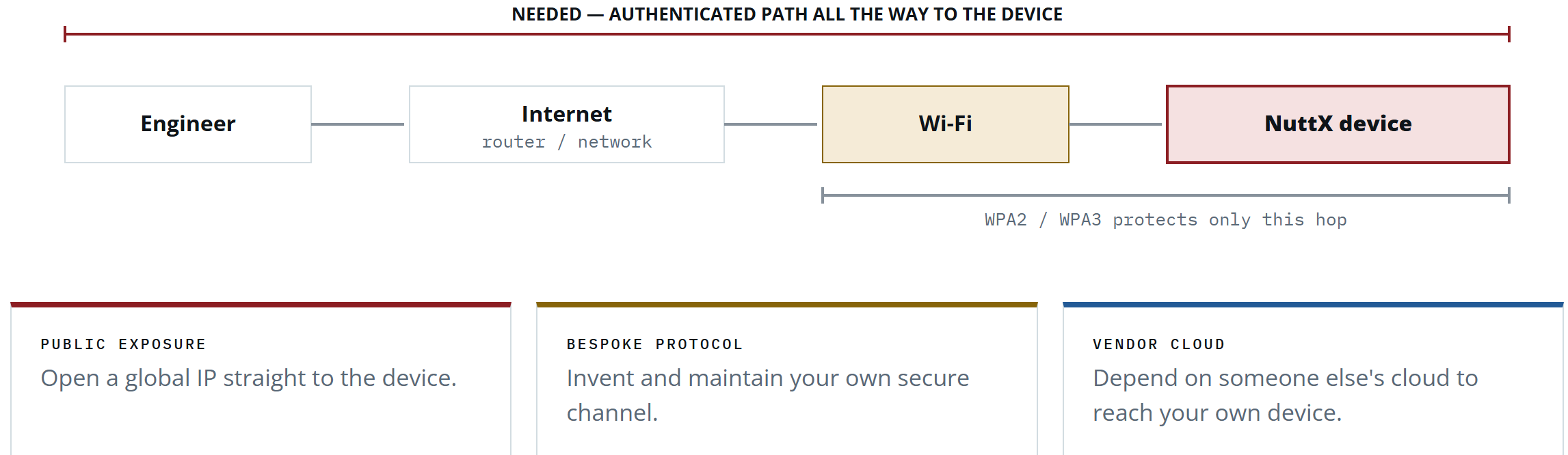
Edge AI · camera deployment · ESP32 + NuttX · operation in the field, not on a bench.

**My perspective: application development → system integration → deployment & operation.**

**TAKEAWAY** My problem was not getting NuttX to run — it was reaching NuttX devices after they were deployed.

# Problem

## The remote access gap.

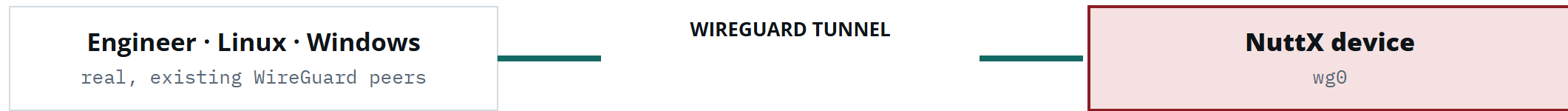


**TAKEAWAY** The missing piece was a standard authenticated network path all the way to the NuttX device.

# Solution

## Why WireGuard fits this problem.

---



- **Modern IP-layer VPN**

small, simple design — a fraction of the codebase of IPsec or OpenVPN.

- **UDP transport, public-key peers**

no certificate authority or session negotiation to operate.

- **Already everywhere**

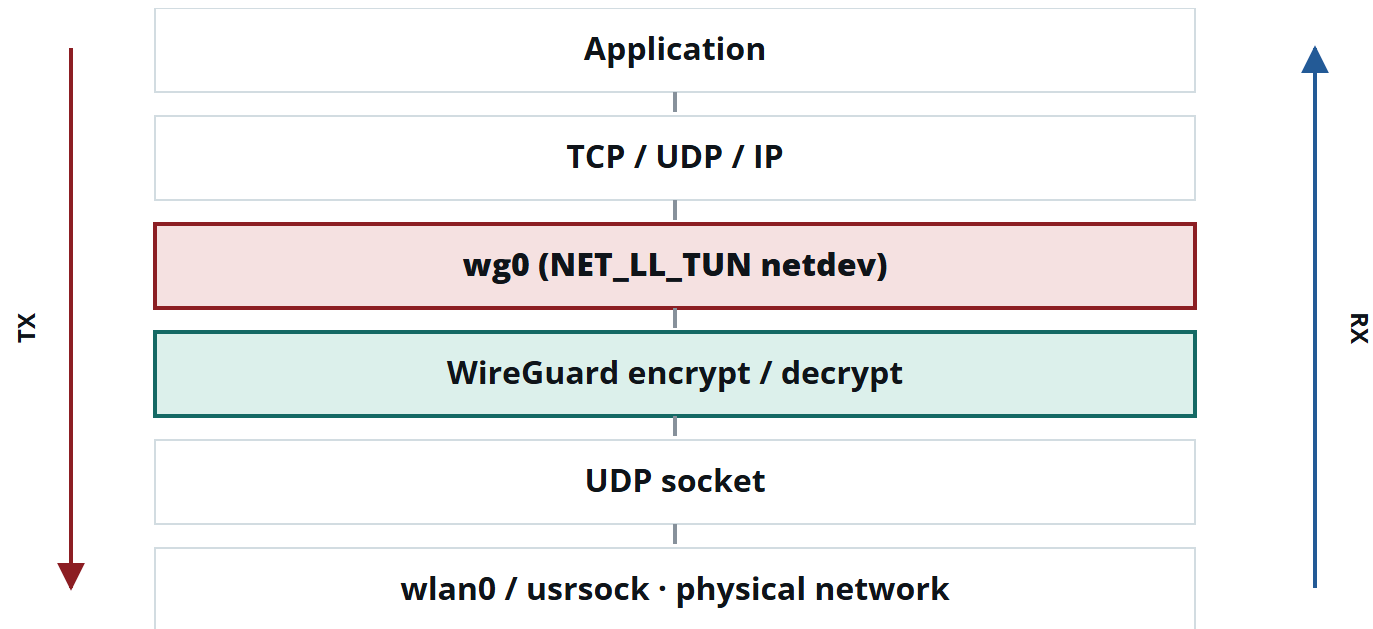
mainline Linux kernel, an official Windows client, interoperable with existing endpoints — not a bespoke protocol.

---

**TAKEAWAY** WireGuard solves the access problem with a standard protocol instead of a bespoke security layer or proprietary cloud.

# Architecture

## WireGuard on the NuttX network stack.



**TAKEAWAY** To NuttX, wg0 is an ordinary network device; WireGuard turns its packets into encrypted UDP underneath.

# Agenda

Seven chapters, about thirty minutes.

---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# Porting Strategy

## Reusing wireguard-lwip without lwIP.

### PROTOCOL & CRYPTO — KEEP

Handshake, transport, BLAKE2s, ChaCha20-Poly1305, Curve25519. Portable C, no OS dependency. Used as-is.

### PLATFORM HOOKS — ADAPT

All OS-specific behaviour isolated behind `wireguard-platform.h` — four functions: clock, random bytes, TAI64N time, load signal.

### LWIP NETWORK GLUE — REPLACE

`wireguardif.c`'s lwIP `netif` registration and buffers. NuttX has no lwIP; this layer becomes the actual porting work.

**TAKEAWAY** A clean platform boundary let the protocol and crypto move unchanged; only the OS and network glue had to be replaced.

# netdev Design

## Mapping lwIP onto NuttX netdev.

| CONCEPT                | LWIP         | NUTTX                      |
|------------------------|--------------|----------------------------|
| <b>NIC abstraction</b> | struct netif | <b>struct net_driver_s</b> |
| <b>Buffers</b>         | pbuf         | <b>iob</b>                 |
| <b>Registration</b>    | netif_add()  | <b>netdev_register()</b>   |
| <b>Send trigger</b>    | callback     | <b>devif_poll()</b>        |

`wg0` is a `NET_LL_TUN` netdev, and its wire is a UDP socket.

**TAKEAWAY** Porting meant mapping onto NuttX's native abstractions, not carrying lwIP into NuttX.

# Result

A working tunnel is only the beginning.

| LAYER                        | PORTING COST             | IN THIS PORT                                  |
|------------------------------|--------------------------|-----------------------------------------------|
| <b>Protocol &amp; crypto</b> | None — portable C, as-is | <b>3,079 lines byte-identical to upstream</b> |
| <b>OS hooks</b>              | Low — four functions     | <b>clock, /dev/urandom, TAI64N, load</b>      |
| <b>Network glue</b>          | The actual work          | <b>netdev + UDP socket + RX task</b>          |

**It worked:** `wg genkey / set / setconf` **at run time — no rebuild to change keys or peers.**

And that is exactly where the interesting part begins — because working was the easy part.

**TAKEAWAY** Getting a tunnel up was the cheapest milestone; almost everything interesting came after it already “worked.”

# Agenda

Seven chapters, about thirty minutes.

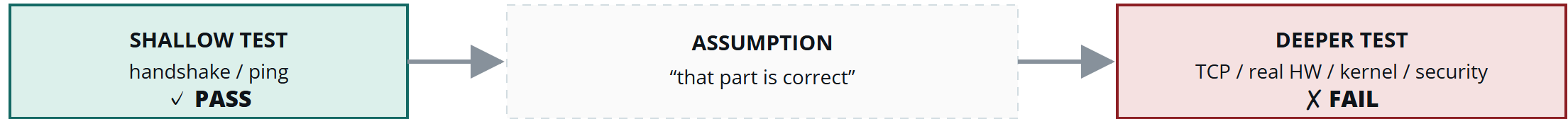
---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# The Testing Pattern

Shallow tests pass, deeper tests fail.

---



## A shallow test passes. A deeper one fails.

And because the shallow test passed, you trust that area and spend hours looking somewhere else. That loop repeated all the way through this project — and it did not stop at the apps version.

# Pitfall 1 / 7

A return code of 0 that did nothing.

---

`setsockopt(SO_RCVTIMEO)` returns **0 — success**.

But `recvfrom()` still **blocks forever**. The timeout was accepted and ignored.

## A perfect chicken-and-egg:

timer never fires → no handshake initiation is sent  
→ the peer stays silent → `recvfrom()` never returns.

---

**TAKEAWAY** A successful `setsockopt()` return does not prove the option took effect — and an unhandled option can hide behind a deadlock instead of an obvious error.

## Pitfall 2 / 7

Created successfully. Did not survive.

---

The RX thread was started with `pthread_create()` + `pthread_detach()`.

When the `wg` command exited, the thread **stopped** — not even visible in `ps`.

A `usleep()` right after create made the thread's `printf` appear — so it **was** created.

My mistake: I read `detach` as “outlives the command.” It only controls **when the thread's resources are reclaimed** — not whether it survives its launcher exiting.

---

**TAKEAWAY** `pthread_detach()` controls resource reclamation, not lifetime — a detached thread still dies with the task group that launched it.

# Pitfall 3 / 7 • the good one

ping works. TCP dies.

---

Same tunnel: `ping` is **0% loss**. A telnet 3-way handshake even **completes** — then the connection dies.

```
wg_encrypt_and_send: psock_sendto  
ret=-1 errno=9 EBADF
```

Sends that **worked** ran in the `wg_rx` task's own context. Sends from another task got EBADF.

**Cause:** fds are scoped to the **task group**, not global.

**Fix:** the fd-less internal API — `psock_*`() on a `struct socket`, usable from any task.

---

**TAKEAWAY** In NuttX, file descriptors are scoped to the task group, not global — a socket created in one task's fd table is not usable by another task's fd, even against the same kernel object.

## Pitfall 4 / 7

“No real impact” was 10×.

---

**The fd-less fix broke `poll()` too. Stopgap: poll every 50 ms.**

Assumption: “it just wakes the CPU when idle.” Wrong — arriving packets waited up to 50 ms, and that capped **throughput by 10×**.

**Fix: hook `psock_poll()` with a callback and a semaphore — real blocking wait.**

---

**TAKEAWAY** The 50 ms polling workaround preserved correctness but reduced throughput by roughly an order of magnitude — an unmeasured stopgap can look free and cost 10×.

# Pitfall 5 / 7

sim passes. Hardware freezes.

---

Config persistence via

`wg showconf > /data/wg0.conf` froze the board — only on the **SPIFFS of real hardware**, never in sim.

The `wg_rx` task calls `ipv4_input()` on **its own stack** — the whole TCP depth rides on it, plus reply encryption.

Default **3072 B** was a number nobody had measured. Found with `CONFIG_STACK_COLORATION + ps`.

---

**TAKEAWAY** A task's default stack size is a guess until it's measured on the hardware that will actually run the deepest call path — sim's generous stacks hide the deficit.

# Agenda

Seven chapters, about thirty minutes.

---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# Part 3 • Going deeper

## Into the kernel.

---

### The limit of version 1

It lives in `apps/` and leans on `psock_*` internals.  
Fine for a FLAT build — but it cannot be a first-class in-tree driver, and it does not fit **PROTECTED / KERNEL** builds, where apps and kernel are separated.

### The move

Put the device **in the kernel** —  
`drivers/net/wireguard/` — and drive it from a small user-space `wg` command over **ioctl**.

---

**TAKEAWAY** An apps/-space port proves the protocol; only an in-tree kernel driver is a first-class device that survives PROTECTED and KERNEL builds.

# The Kernel Design

A kernel netdev, an ioctl ABI, a write-only key.

---

- **Kernel netdev**

`netdev_lowerhalf` owning a UDP socket, a receive kernel thread, timers and cookies — the driver owns the socket and thread, so v1's launcher-lifetime dependency is gone.

- **Flat, pointer-free ioctl ABI**

`SIOCS/GWGIF`, `SIOCS/D/GWGPEER` — no pointers, because in PROTECTED / KERNEL the kernel copies the caller's struct directly.

- **Private key is write-only**

the get ioctl never returns the private key.

- **Kernel crypto is NuttX's own**

BLAKE2s, ChaCha20-Poly1305, Curve25519 from `crypto/` — nothing vendored on the kernel side. The one vendored file, MIT X25519, is in the user-space command's offline `genkey` — reaching the kernel's Curve25519 via cryptodev instead would pull in the whole software cipher suite for one scalar multiply.

---

**TAKEAWAY** The socket, the thread, and the pointer-free ABI are one fix: don't trust code that can exit, or a boundary it can't cross.

# Two more bugs · same shape

Deeper depth, same failure.

---

## A bug in NuttX's own crypto

`chacha20poly1305` put the counter in the wrong nonce bytes. Counter 0 matches, so **the handshake succeeds and every data packet after the first fails**. No in-tree caller had ever exercised it. Two-line fix, going upstream as a separate `crypto:` PR.

## A BUILD\_KERNEL-only crash

`wg_set_if()` snapshotted peers into a **6 KB array on the stack**; the kernel stack is 3 KB → overflow → heap corruption → panic. **sim (FLAT, big stacks) never hit it**. Moved to the heap.

---

**TAKEAWAY** A successful WireGuard handshake does not prove that data-path crypto or kernel behavior is correct — both bugs here only surfaced past the handshake, on a real kernel build.

# Agenda

Seven chapters, about thirty minutes.

---

|   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# Verification

Seven places it has to hold.

| ENVIRONMENT           | IMPLEMENTATION           | CONFIRMED                                                                                              |
|-----------------------|--------------------------|--------------------------------------------------------------------------------------------------------|
| sim:wireguard         | kernel driver            | builds clean (the config CI compiles)                                                                  |
| rv-virt knetnsh64     | kernel · BUILD_KERNEL    | virtio-net tunnel to Linux; wg a separate ELF across the syscall boundary                              |
| rv-virt pnsh64        | kernel · BUILD_PROTECTED | tunnel with the MPU split live — wg in nuttx_user , every ioctl across the boundary, no fault          |
| rv-virt knetnsh64_smp | kernel · SMP, 4 CPUs     | a flood concurrent with reconfiguration and a down / up — no fault, tunnel recovered                   |
| SPRESENSE, measured   | kernel · Cortex-M4F      | under load and 40 down / up cycles: heap, buffers and stacks identical — receive thread 1472 B of 6096 |
| ESP32-S3 + SPRESENSE  | kernel · real Wi-Fi      | tunnel to the Windows client — ESP32 native 4/4, SPRESENSE via usrsock 6/6                             |
| same two boards       | apps v0.1.1              | Linux & Windows peers; telnet, HTTP, 7 MB, rekey, power-cycle                                          |

Both boards, both privilege models, more than one CPU — and the resources measured, not assumed.

**TAKEAWAY** A tunnel in sim isn't verification — it must survive both privilege models, multiple CPUs, and hardware measurement.

Hardware runs used the feature revision before the final upstream rebase; KERNEL/SMP tunnels were rerun on the rebased submission candidate.

# Verification · the limits

A working tunnel is not evidence.

| FAILURE                                                                                                                          | DOES A PASSING PING NOTICE?             | WHAT WAS MEASURED                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The same private key on every boot — the xorshift128 <code>/dev/urandom</code> backend can use compile-time seed material</b> | no — every functional test still passes | <b>3 restarts → 3 different keys; a sanity check of the pool path, not entropy certification</b>                                                                                                                                     |
| <b>Session keys outliving the tunnel</b>                                                                                         | no                                      | <b>read the live process memory: transport keys 32/32 bytes set while up → all zero after <code>down</code> ; exactly one copy of the static key, before and after</b>                                                               |
| <b>A leak that only shows after hours</b>                                                                                        | no                                      | <b>151 forced rekeys past the expiration horizon, 10 endpoint moves, 100 <code>down</code> / <code>up</code> : buffer pool whole, live allocations back to the boot count exactly, RX-thread stack high-water 2.5 KB and unmoved</b> |

**Each of these passes every functional test while being broken.**

**TAKEAWAY** A working tunnel proves none of: unique keys, wiped secrets, or stable resources — each needs its own targeted check.

# A design call

## The timestamp problem has no free answer.

A responder drops any handshake timestamp at or below the greatest it has accepted. So the initiator must beat its own past — **across reboots**.

| OPTION                               | DURABLE WRITES  | NO RTC | CALL                                          |
|--------------------------------------|-----------------|--------|-----------------------------------------------|
| <b>Realtime + in-boot high-water</b> | none            | no     | <b>adopted</b>                                |
| <b>Persist every timestamp</b>       | every handshake | yes    | flash wear, on the handshake path             |
| <b>Durable range reservation</b>     | one per N       | yes    | needs a durability contract NuttX has not got |
| <b>Let it catch up</b>               | none            | no     | the wait equals the last uptime               |

On the board, one variable changed, nothing sent to it: **clock unset** → **no handshake in 75 s**; **clock set** → **4.1 s**.

**Took the contract Linux and wireguard-go use — and made the case it misses **say so**.**

**TAKEAWAY** No RTC, no timestamp progress: a device can be unable to dial out and still pass a ping test.

# Agenda

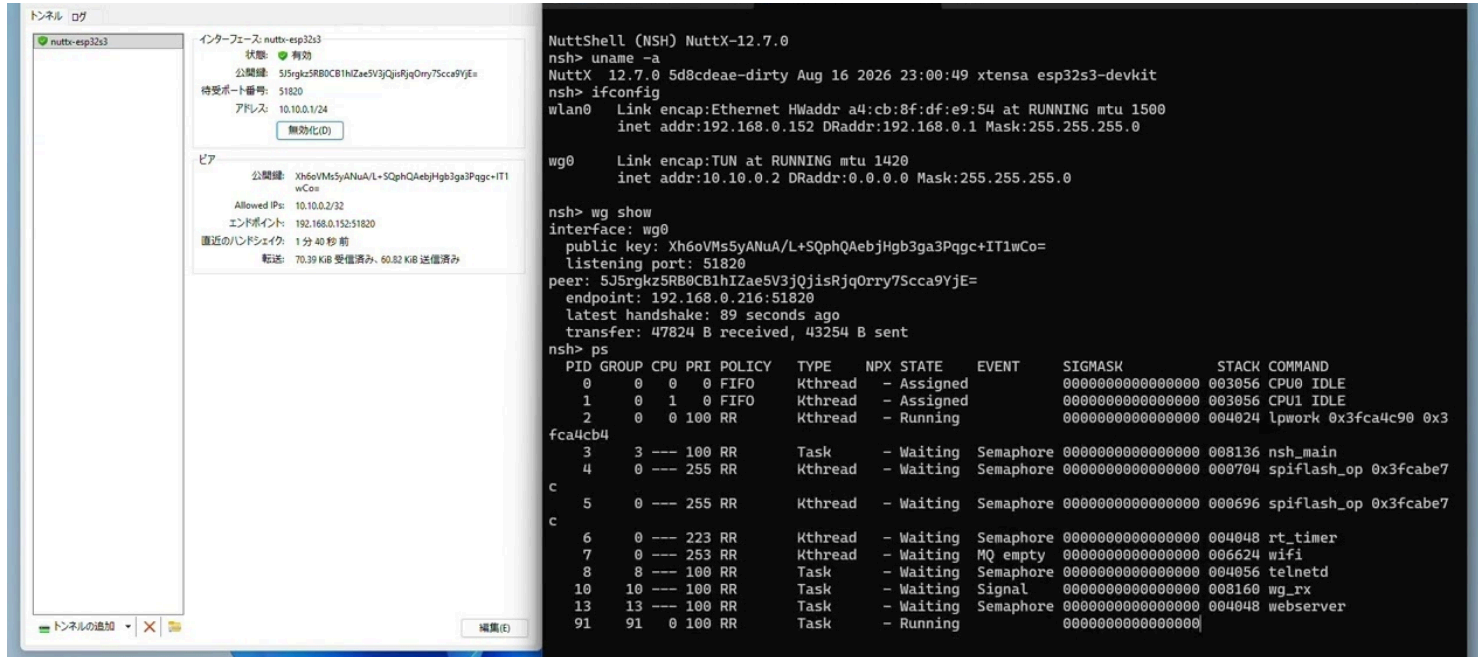
Seven chapters, about thirty minutes.

---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# The Demo

An ordinary host at the other end of the tunnel.



telnet into the board **through the tunnel** and run commands. Start a web server on the board and open it in a browser.

USB unplugged — the board runs on a power adapter alone. The peer is the **official Windows WireGuard client**, over home Wi-Fi.

[youtu.be/1kyX2av5WG4](https://youtu.be/1kyX2av5WG4)

# Agenda

Seven chapters, about thirty minutes.

---

- |   |                                                                                                                     |         |
|---|---------------------------------------------------------------------------------------------------------------------|---------|
| 1 | <b>Background</b>                                                                                                   | ~6 MIN  |
|   | What NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard. |         |
| 2 | <b>Porting Strategy</b>                                                                                             | ~2 MIN  |
|   | Starting from wireguard-lwip and mapping it onto NuttX's own netdev.                                                |         |
| 3 | <b>Hidden Pitfalls</b>                                                                                              | ~4 MIN  |
|   | Five bugs a working demo never revealed — two more wait in the kernel.                                              |         |
| 4 | <b>Kernel Driver</b>                                                                                                | ~2 MIN  |
|   | Moving <code>wg0</code> from apps/ into a real in-tree kernel device.                                               |         |
| 5 | <b>Verification</b>                                                                                                 | ~11 MIN |
|   | Seven environments, what a passing ping can't tell you, and one design call.                                        |         |
| 6 | <b>Demo</b>                                                                                                         | ~2 MIN  |
|   | An ordinary host, reached through the tunnel, on real hardware.                                                     |         |
| 7 | <b>Toward Upstream</b>                                                                                              | ~3 MIN  |
|   | Operability, portability, contributing back, and what to carry out.                                                 |         |

# Operability

From a demo toward something operable.

|                      | BEFORE                                | AFTER                                                                       |
|----------------------|---------------------------------------|-----------------------------------------------------------------------------|
| <b>Keys</b>          | baked into Kconfig; rebuild to change | <b>runtime genkey / set / setconf; never in the build artifact</b>          |
| <b>When it stops</b> | guesswork                             | <b>sample ROUTER / LAN / TUN / TCP every minute — say whose fault it is</b> |

**Four independent signals answer one question: when the tunnel goes quiet, is it me?**

**TAKEAWAY** Runtime keys and layer-by-layer health signals turn "the tunnel is down" into "the router side is down" — the difference between a demo and something you can run unattended.

# Portability

The “four functions” bet, checked.

The apps (FLAT) implementation, moved across four architectures with **no code change**:

| TARGET                            | CODE CHANGE | RESULT                                  |
|-----------------------------------|-------------|-----------------------------------------|
| <b>x86_64 (sim)</b>               | none        | <b>tunnel</b>                           |
| <b>ARM Cortex-A7 (QEMU)</b>       | none        | <b>tunnel</b>                           |
| <b>Xtensa LX7 (ESP32-S3)</b>      | none        | <b>tunnel over real Wi-Fi</b>           |
| <b>ARM Cortex-M4F (SPRESENSE)</b> | none        | <b>tunnel over real Wi-Fi (GS2200M)</b> |

**TAKEAWAY** Four functions of OS glue carried one source across four architectures, unchanged — the porting bet paid off.

Verified with the apps (FLAT) implementation; the kernel driver adds sim, rv-virt (BUILD\_KERNEL) and both boards over real Wi-Fi (ESP32-S3 native, SPRESENSE over usrsoc) on top.

# Contributing Back

## The Apache Way.

---

- **A small `crypto`: PR first**

the ChaCha20-Poly1305 nonce fix — a bug fixed *for* the project, on its own.

- **Then the driver PR**

`net/wireguard` — the kernel device, ABI, and its documentation.

- **Then the command PR**

`apps/system/wg` — the ioctl client.

- **Design prepared for `dev@nuttx.apache.org`**

checkpatch / nxstyle clean, PR-shaped, licensed — community over code.

---

**TAKEAWAY** Splitting the crypto fix into its own PR lets it land in NuttX on its own merits, independent of whether the driver itself is accepted.

Status: three signed candidate branches are published on the author's forks; upstream discussion and PRs are not open yet.

# Takeaways

Three things to carry out.

---

- **Shallow tests pass, deep tests fail**

design tests for depth — sustained data, real hardware, a real kernel build — not just “it handshook.”

- **A clean OS-abstraction bet pays off**

four architectures, three versions, no changes.

- **Porting into a project can improve the project**

two latent NuttX bugs found on the way in — its ChaCha20-Poly1305 nonce, and a boot regression that stops a clean release from reaching a shell on this board at all.

COMMUNITY OVER CODE · GLASGOW 2026

# Thank you

Questions welcome — the kernel driver, the bugs, the ioctl ABI, or the hardware.

[youtu.be/1kyX2av5WG4](https://youtu.be/1kyX2av5WG4)

DEMO

[apache/nuttx](#) · [apache/nuttx-apps](#)

HEADING UPSTREAM

Satoru Akita · Sony Semiconductor Solutions

